

RecruitAir

MLOPS project milestones 1-3

[Criteria extractor repository](#)

[Criteria evaluator repository](#)

Made by:

- Alfonso Brown
- Tania González
- Virginia Nicosia
- Marc Parcerisa
- Daniel Reverter

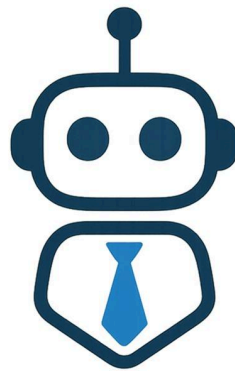


Table of contents

Introduction	2
Goal of the project	2
Teammates' evaluation	3
Methodology	4
Milestone 1: Inception	4
Selection of problem and requirements engineering for ML	4
Dataset card	7
Model card	7
Project coordination and communication	8
Selection of the cloud provider	9
Milestone 2: Model Building – Reproducibility	9
Project structure	9
Code versioning	10
Data versioning	11
Experiment tracking	13
Milestone 3: Model Building – Quality Assurance	13
Energy efficiency awareness	13
Static code analysis	13
Model testing	13
Data testing	14

Introduction

Goal of the project

The goal of this project is to develop **RecruitAIr**, an AI-driven recruitment system designed to automate and enhance the candidate evaluation process for specific job offers.

Manually reviewing and filtering large numbers of job applications is inefficient and time-consuming. Automating this process enables recruiters to focus on the most relevant candidates and make faster, more objective hiring decisions.

However, most existing applicant management systems rely heavily on fully automated ranking algorithms that attempt to predict the best candidates based on their perceived fit to a given job description. While this approach can save time, it often removes the recruiter's involvement from the earliest and most critical stages of the selection process. By delegating candidate evaluation entirely to the system, recruiters lose visibility and control over why certain applicants are prioritized over others. These systems tend to act as opaque *black boxes*, making decisions that are difficult to interpret or adjust. Moreover, automated ranking often overlooks contextual nuances such as company culture, team dynamics, or specific non-technical requirements that only a human recruiter can properly assess. As a result, important qualitative aspects of candidate evaluation are ignored, and the process becomes overly dependent on the biases or limitations of the algorithm rather than the informed judgment of the recruiter.

The proposed system leverages artificial intelligence not to replace the recruiter's judgment, but to **assist and streamline** the evaluation process while maintaining full transparency and human control. When a new job offer is imported, the system can automatically analyze the description and suggest a set of key evaluation criteria, such as required skills, experience levels, or language proficiency, along with estimated importance weights for each. These automatically generated elements serve only as **initial recommendations** to save time, the recruiter can freely review, modify, add, or remove any criterion or weighting at any point in the recruitment process. Once a set of criteria is decided upon, the system evaluates each applicant in relation to these clearly defined factors, producing a structured overview rather than an opaque ranking. This approach allows recruiters to see exactly **how and why** each candidate received a particular score, enabling data-driven insights without ever losing human oversight or decision-making authority.

In this sense, **RecruitAIr** provides recruiters with:

- A user interface to manage and validate job offers, automatically transforming them into structured, actionable criteria lists, which can be modified at any point.
- Secure applicant registration and data handling, ensuring compliance with privacy and data protection regulations.
- Automated evaluation and ranking of candidates according to each and every criterion, presenting detailed match scores and allowing recruiters to adjust weights, filter results, and provide feedback to continuously improve the models' performance.

Teammates' evaluation

☒ “All team members agree that they had an equal contribution to this delivery and project: completing a fair share of the team's work with acceptable/high quality, keeping commitments, and completing assignments on time, helping teammates who are having difficulty when it is easy or important”.

Methodology

Milestone 1: Inception

Selection of problem and requirements engineering for ML

To address the identified challenges while maintaining transparency, flexibility, and human oversight, the system architecture was designed around a clear separation of responsibilities. Following the project's guiding business values, the solution is divided into three main components: the **application layer** and two independent **AI subsystems**. The application layer encompasses all user-facing functionalities, including the interface, security, and workflow management, ensuring a seamless and controlled user experience while maintaining absolute governability over the personally-identifiable data. The two AI modules, on the other hand, are conceived as autonomous services that can be developed, trained, tested, and deployed independently, facilitating scalability, maintainability, and team specialization.

The first subsystem, the **Criteria Extractor**, processes job offer descriptions to identify and structure the key decision factors. It outputs standardized JSON objects containing concise, actionable criterion descriptions along with recommended importance weights. For the first version of the system, this system was chosen to be implemented using open source LLMs with *structured output* capabilities.

The second subsystem, the **Criteria Evaluator**, takes an individual criterion and an applicant's résumé as input and returns a relevance score from 0 to 10, indicating how well the candidate fits that specific requirement. This modular design ensures that each component has a well-defined purpose and can evolve independently while maintaining full compatibility within the overall RecruitAIR ecosystem. Due to the peculiar nature of this system, no general open source models existed that served its purpose, which meant that this model is to be designed and trained by our team, preferably leveraging transfer learning techniques and existing open source small LLMs.

We separated the functional and non-functional requirements by their scope: [General Application Requirements](#), [Criteria Extraction Requirements](#), and [Criteria Evaluation Requirements](#).

Dataset card

For this project, we selected three existing datasets that support different aspects of job–applicant matching and AI-based recruitment analysis:

- **Job Skill Set (Kaggle):** Contains detailed information on job roles, descriptions, and associated skills. It is ideal for developing models related to job matching, skill extraction, and career recommendations.
- **Recruitment Data Set (Kaggle):** A synthetic and anonymized collection of job applications and descriptions designed to simulate real-world hiring scenarios, useful for data science and HR analytics experiments.
- **Resume Score Details (Hugging Face):** Includes 1,031 samples of resumes and job descriptions evaluated with GPT-4o to measure alignment, skill matching, and job fit scoring based on predefined criteria.

The dataset cards can be found attached in the following [link](#).

Model card

Two model cards were created, one for each of the main ML components of RecruitAIr; each model card follows the [Hugging Face Model Card Specification](#).

RecruitAIr_JobCriteriaExtractor

- **Goal:** Automatically extract structured criteria (skills, experience levels, and qualifications) from job descriptions.
- **Architecture:** Transformer-based LLM (Dolphin3, via Ollama).
- **Inputs / Outputs:** Takes job text and returns a list of structured criteria (title, description, importance score).
- **Training Data:** Combination of datasets from Kaggle and Hugging Face (Job Skill Set, Recruitment Dataset, Resume Score Details).
- **Evaluation:** Target recall = 0.329, Importance MSE 2425.68
- **License:** MIT.
- **Limitations:** Performance decreases in highly specialized domains; all outputs must be reviewed by recruiters to mitigate bias.

The full model card can be found [here](#)

RecruitAIr_CriteriaEvaluator

- **Goal:** Evaluate how well each applicant matches a specific job's required criteria.
- **Architecture:** Transformer-based causal language model (Qwen3).
- **Inputs / Outputs:** Takes a candidate's resume and a job's criteria and outputs a compatibility score and explanation.
- **Training Data:** Derived from resume–criteria pairs in the Resume Score Details dataset.
- **Evaluation:** Training MSE 0.00054, Validation MSE 0.00358
- **License:** MIT.

- **Limitations:** Works only in English and may favor keyword-heavy resumes; not intended for automated hiring.

The full model card can be found [here](#)

Project coordination and communication

To ensure smooth collaboration, transparency, and progress tracking across all milestones, the team adopted a mix of synchronous and asynchronous tools:

- Communication channels
 - WhatsApp group: used for fast coordination, quick updates, and short-term task allocation. Ideal for daily communication and resolving blockers quickly.
 - Discord server: set up for structured team communication and voice meetings. Discord also serves as our space for informal brainstorming and documentation sharing during development.
- Version control and collaboration
 - Github: central to our workflow, hosting all repositories for the RecruitAir project.
- Documentation and Traceability
 - Reports folder: contains milestone deliverables (dataset cards and model cards).
 - Commit history: all commits are descriptive and trace back to specific tasks or discussions on GitHub or Discord, ensuring traceability and accountability.
 - Experiment Tracking: MLFlow hosts all the models, prompts and experiments performed throughout the development of the application, properly versioned and organized.
 - Artifact Serving: Docker images containing the application and APIs will be stored in GitHub's Container Registry ([ghcr.io](#)), making them accessible across the world for fast deployment. Images will be labelled with unique, immutable tags (to the extent possible).
- Coordination practices
 - Weekly checkpoints: the team holds short weekly syncs via Discord to review progress, assign next steps, and ensure alignment.
 - Individual tasks are distributed, monitored and followed from inception to completion using GitHub Projects, as seen below.

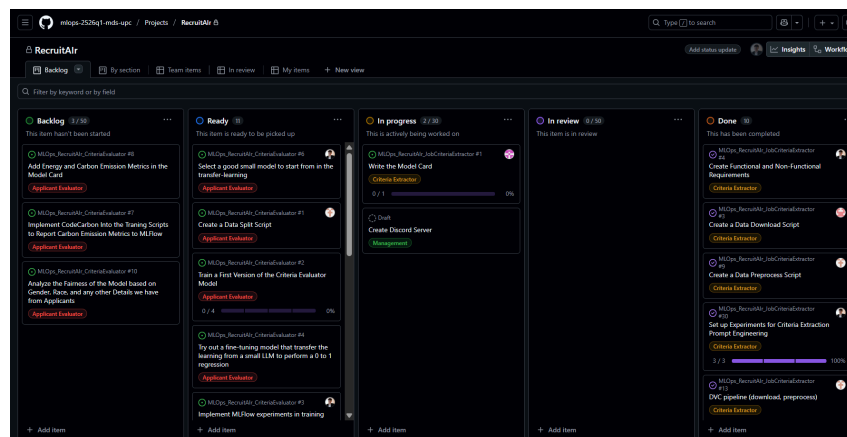


Figure 1: GitHub Project for milestones 1-3. Screenshot taken 2 weeks before delivery

Selection of the cloud provider

The application has been designed to be completely modular, and processes communicate with one another over HTTP requests. This means that all three of the components can be deployed, if necessary, in different cloud environments. However, this wouldn't be practical. For that reason, we decided upon a single cloud provider: AWS.

This decision was made due to two reasons: The first reason is that some in the team already have experience with AWS, and know most of the capabilities that it offers, even allowing for hosting AI systems natively (See SageMaker). The second reason is that we considered using UPC's Virtech, but the limited quota and hardware capabilities rapidly convinced us otherwise (For instance, we can only deploy one VM at a time, which wouldn't make sense in our modular design).

Nonetheless, because we needed an MLFlow server to track the experiments, and we needed it to have the latest prompt-engineering capabilities, we ended up using the Virtech cloud for hosting the tracking server.

Milestone 2: Model Building – Reproducibility

Project structure

The team wanted to design a structure that would make the code easy to maintain, scalable for future developments, and clear for all current (and future) contributors. To achieve this, we decided to split the work across three separate GitHub repositories, each serving a distinct purpose within the system's architecture:

1. **MLOps_RecruitAlr_JobCriteriaExtractor**: Dedicated to the first model, which extracts job requirements from job offers using a Large Language Model (LLM).
Repository: https://github.com/mlops-2526q1-mds-upc/MLOps_RecruitAlr
2. **MLOps_RecruitAlr_CriteriaEvaluator**: Contains the second model, which evaluates the fit between a candidate and a specific job requirement.
https://github.com/mlops-2526q1-mds-upc/MLOps_RecruitAlr_CriteriaEvaluator
3. **API and Web Repository** (to be created in Phase 2): Will host the web or API interface connecting both models for deployment and user interaction.

From an MLOps perspective, separating repositories provides several advantages:

- Each repository focuses on a single component. This prevents dependency overlap and makes debugging, testing, and refactoring more manageable.
- Each component has its own lifecycle, so they can be deployed, versioned, or scaled independently.
- Different team members can work in parallel without conflicts, and issues or pull requests remain tied to specific subprojects.
- Modular repositories limit the risk of entangling data, models, and infrastructure, reducing future technical debt.

The two model repositories follow the Cookiecutter Data Science structure. It provides a standardized folder organization that promotes clarity and reproducibility. Using this template ensures:

- A consistent layout across projects (data/, notebooks/, src/, models/, etc.) that makes it easier to visualize and reproduce for new collaborators.
- Clear separation between raw, interim, and processed data.
- Easier automation in future CI/CD workflows, as the directory structure follows recognized conventions.

This structure makes our project clean, maintainable, and ready for scaling and deployment.

Code versioning

To maintain code quality and enable collaboration, our team adopted the GitHub Flow workflow. This branching strategy allows us to build the repositories quickly while keeping the main branch stable.

Our workflow follows these steps:

1. Each new feature or fix starts from the main branch, which is protected against direct pushes.
2. Work is done in a feature branch, named after the specific task (for example `data_preprocess`, `criteria_extraction`, `prompt_engineering`, or `linting`).
3. When a task is finished, a Pull Request (PR) is opened to merge the branch back into main.
4. The PR is reviewed by at least one teammate before merging, ensuring code quality and mutual understanding of changes.

We used GitHub's review tools for inline comments and Discord for quick discussions when needed. The following good practices were followed as much as possible:

- **Descriptive commit messages:** Each commit summarizes the purpose of the change in plain language, such as “extract criteria using dolphin3”.
- **Frequent, small commits:** We preferred incremental updates to large ones, which reduces merge conflicts and makes rollback easier.
- **Clean history:** Only reviewed and approved changes are merged into main, keeping the main branch stable.

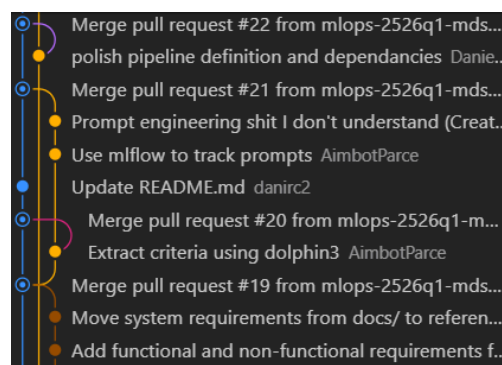


Figure 2: Snapshot of workflow in Job Criteria Extractor

Using Github Flow gives our project transparency, accountability, reduced risk and reproducibility.

Data versioning

To guarantee reproducibility we implemented Data Version Control in both model repositories. DVC enables efficient tracking of data and preprocessing steps without the storage space limitations of GitHub.

Each repository has its own data pipeline:

1. **Raw data download:** The two datasets from kaggle and the main dataset consisting of 1,054 JSON files from a Hugging Face repository are downloaded. The exact SHA code of the commit of the main dataset is recorded in the configuration file to ensure that any teammate can reproduce the exact same version.
2. **Data processing:** Each model requires its own processing. The output of the pipeline feeds directly into either the prompt-engineering process (for the LLM-based model) or fine-tuning (for the numerical-output model).

Importantly, each pipeline stage has its own configuration file, instead of a single global config. This prevents unnecessary reprocessing: when one stage is modified, only that component is recomputed, saving time and computational resources.

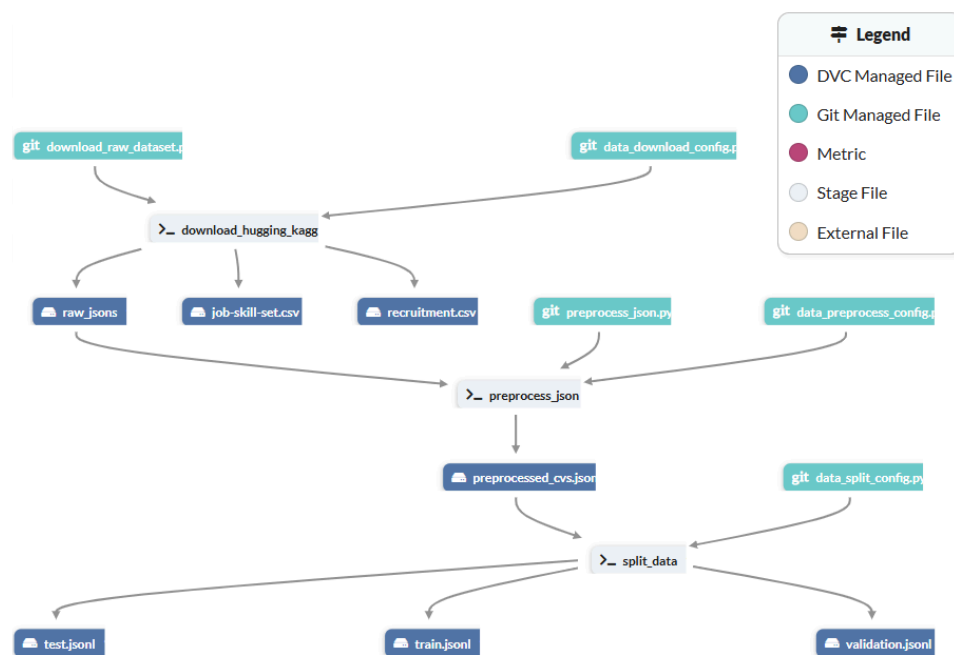


Figure 3: Data pipeline feeding the criteria evaluation model

To support collaboration and remote storage, the team integrated DagsHub with DVC. DagsHub acts as both a remote data repository and a visualization tool for pipeline tracking. This integration allows:

- Synchronization of data and model artifacts across team members.
- A visual overview of pipeline dependencies and version history.
- Combined access to data, code, and experiments in one collaborative platform.

Using DVC and Dagshub this way enables reproducibility, scalability, traceability and collaboration in our data management.

Experiment tracking

Due to the nature of the project, two different experiment tracking strategies had to be implemented:

1. Prompt Engineering Experiments had to be implemented for the Criteria Extraction system. The workflow can be seen in figure 4, and is as follows:

- 1.1. An engineer creates a new prompt version in MLFlow prompt registry, naming it however they please, but tagging it with the tags `domain: recruitair`, `task: criteria-extraction` and `stage: testing`. This is to ensure traceability and governability over all the prompts. This Prompt Version has to be created using a custom script for it, which along with the prompt template, also stores some metadata about the target schema (See [MLFlow Structured Output](#)).
- 1.2. An engineer (or an automatic system) then triggers a prompt evaluation script in a, preferably, GPU-enabled machine. This script receives a set of parameters such as the prompt name and version, or LLM name and version, and executes a testing loop which extracts the criteria and predicted importance from all the job offers in the dataset (see Dataset Cards and Data Versioning). This script evaluates the correctness of the extracted lists based on, mainly, two bespoke metrics: `target-recall`, and `importance-mse` (See Appendix A), as well as several system and emission metrics reported by Codecarbon. It does so by logging a trace for each job offer to MLFlow with the inputs, outputs, targets and metrics, which then can be inspected by an engineer through MLFlow's UI. See figure 5 for a screenshot of the traces page in MLFlow.
- 1.3. The above two steps are repeated until the engineer finds a combination of Prompt and Model with sufficient performance. They then execute a third script which promotes a given prompt into a production registry (inside the same MLFlow registry, a prompt named "criteria-extraction" is reserved for production prompts).
- 1.4. All production systems are built and deployed using the latest versions of the prompts named "criteria-extraction" (the reserved namespace for production prompts, which has the tag `stage: production`).

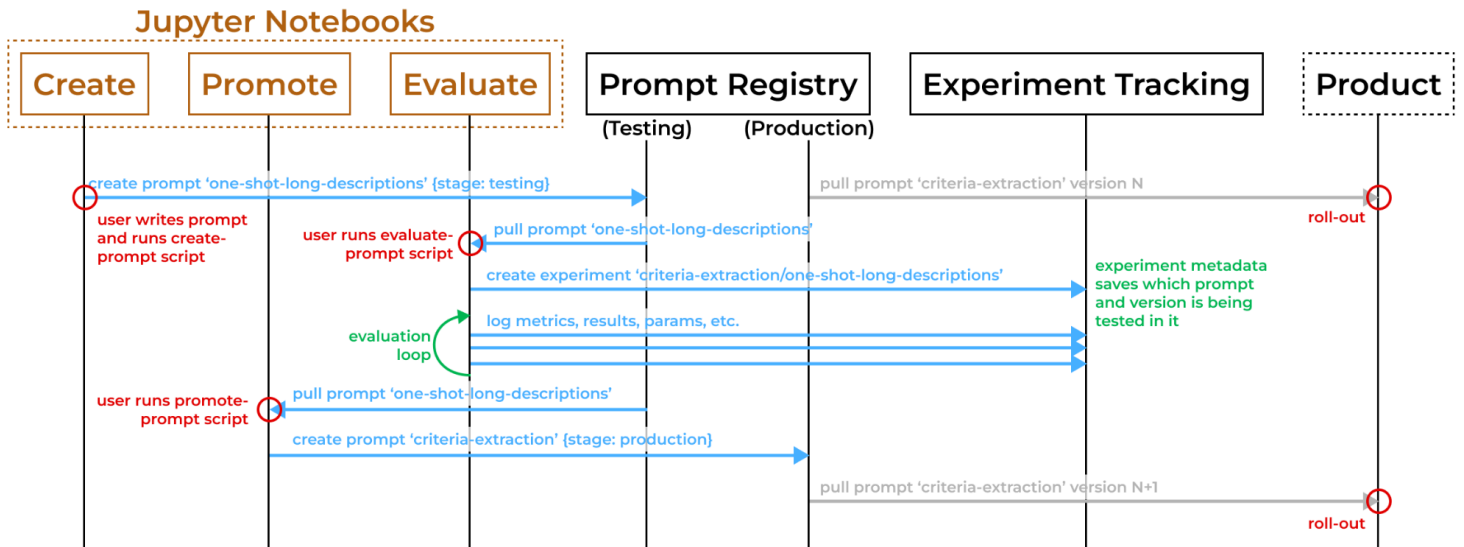


Figure 4: Outline of the life cycle of the Prompt Engineering experiments and their promotion to Production environments for the Criteria Extraction Systems.

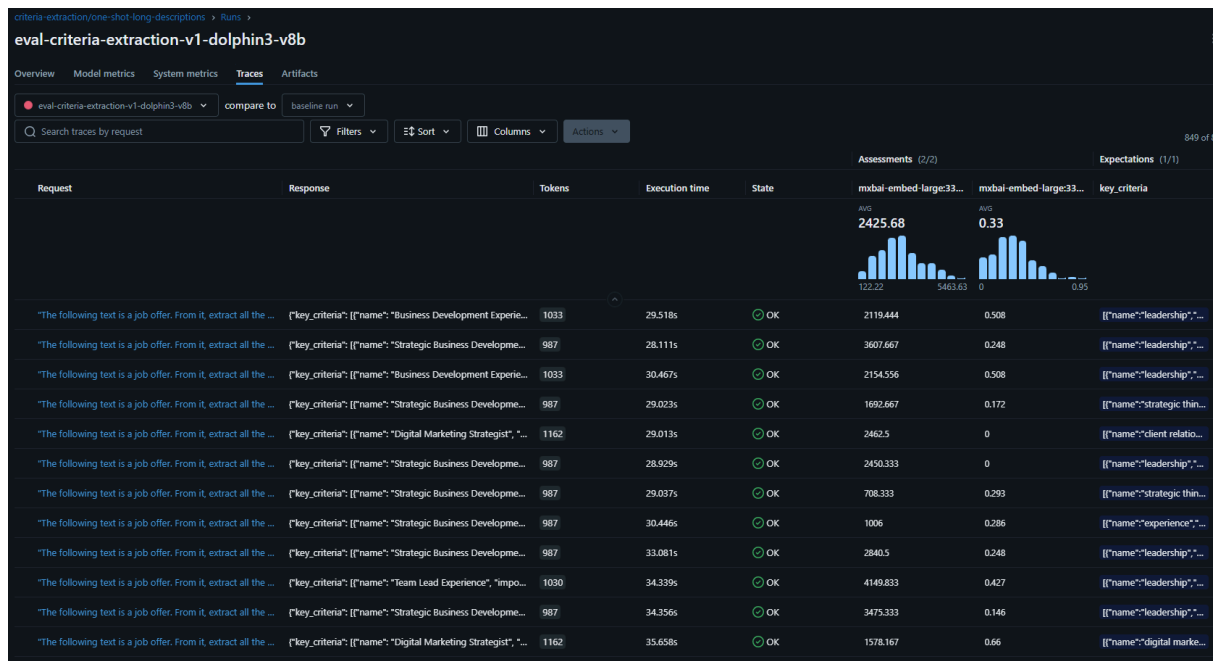


Figure 5: Screenshot of MLFlow's Prompt Engineering Experiment Traces

2. **Traditional LLM-based Model Training Experiments** had to be implemented for the Criteria Evaluation system. The workflow can be seen in figure 6, and is as follows:

2.1. An engineer decides on an architecture and baseline model for performing transfer learning. They explore the model and its associated tokenizer, and define a new untrained model using the strategy that they prefer. In our baseline experiments, Qwen was used as a baseline model, and the fine-tuned model consists of simply swapping the last head of the Transformer-based model for a simple binary classification head (single

output with logistic activation function); and the input of the model is curated by using a custom tokenizer which receives a resume and criteria, tokenizes them, and joins them with *EOS tokens* between the two.

- 2.2. Once they have a proper architecture and transfer learning strategy properly defined, they execute some training script which fine-tunes the model for our target use case. This training script makes use of MLFlow and Codecarbon to constantly log training evaluation metrics, as well as system and emission metrics to MLFlow's tracking server. Finally, at the end of the training loop, an artifact is logged into the tracking server containing the trained model and its tokenizer.
- 2.3. By inspecting the training metrics over MLFlow's UI, the engineer then decides upon a trained model to be promoted to Production. This process, contrary to Prompt Engineering promotions, can be directly done through MLFlow's UI, and simply copies all the artifacts from the model over to a reserved namespace called "criteria-evaluation", also with the tag stage: production.
- 2.4. All products, artifacts and APIs that are to serve criteria extraction models, then pull the latest models from this namespace.

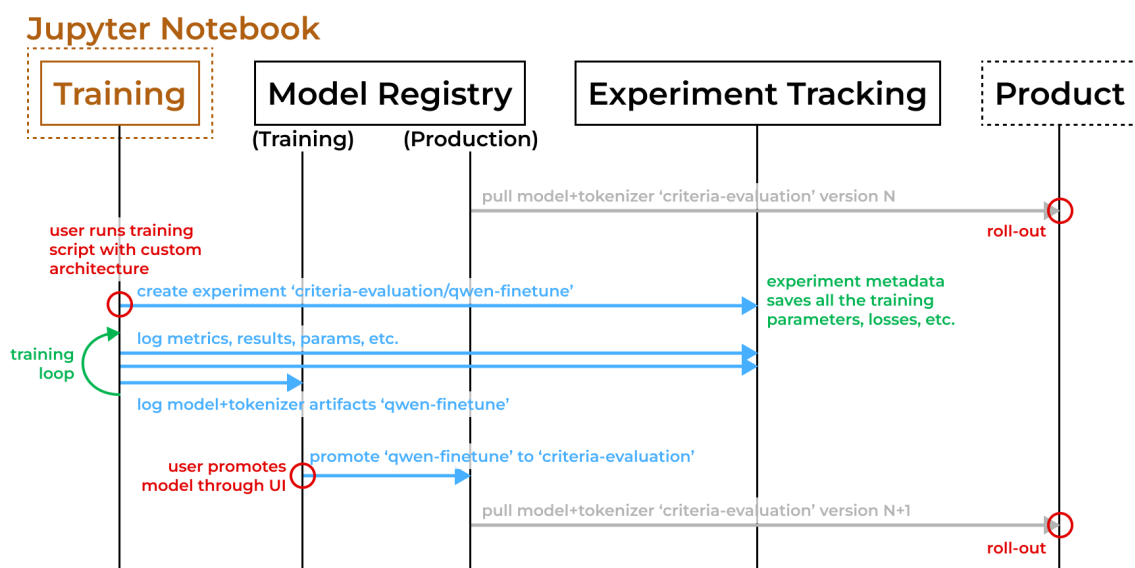


Figure 6: Outline of the life cycle of the Transfer Learning experiments and their promotion to Production environments for the Criteria Evaluation Models.

By implementing these strategies, we achieved a balance between automation, organization, and our principal business value: keeping the human in the loop as much as possible (for instance, having humans evaluating the actual criteria extraction capabilities of the models before promoting them to production is a small tradeoff which ensures that all models are somewhat audited and approved by an engineer).

Milestone 3: Model Building – Quality Assurance

Energy efficiency awareness

Energy efficiency and carbon impact were monitored using CodeCarbon, integrated into all training and prompt engineering experiments. Experiments were conducted locally on an NVIDIA RTX 3060 (1 GPU) workstation with 32 CPU cores and 61.6 GB RAM; the Criteria Extractor consumed approximately 4.6 kWh (≈ 0.8 kg CO₂ eq), while the Criteria Evaluator consumed only 5.6 Wh (<1 g CO₂ eq). These results demonstrate the project's emphasis on green AI practices, ensuring that development remains energy-aware, transparent, and reproducible.

Static code analysis

Static code analysis was introduced to improve code quality and reduce errors. As the project grows, it will become crucial to automate the detection of bugs, stylistic issues, and potential vulnerabilities before they make it into the main codebase. This process ensures that new code meets a consistent standard, which streamlines code reviews and makes the entire system more reliable and easier to maintain.

The system uses several tools for this. Mypy was integrated for type checking, which helps catch type-related errors early. Black, isort, and Flake8 were implemented for consistent code formatting and to identify basic syntax and style issues. Pylint was added to perform a deeper analysis, flagging potential bugs and code smells. To enforce these checks, GitHub Actions workflows were set up for mypy and pylint. These actions run automatically on pull requests, acting as status checks that prevent code from being merged unless it passes all the required static analysis tests.

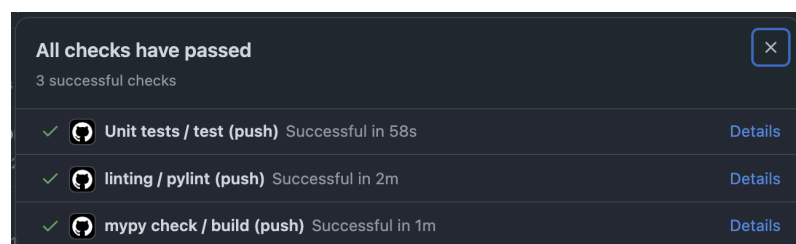


Figure 7: Results of github workflow running after commits have been pushed to the main branch.

Model testing

Adding unit tests using pytest is also essential for validating our system's logic and preventing regressions. Unit tests allow us to test individual functions and components in isolation, ensuring they behave as expected and that any changes to the code haven't introduced new bugs.

We integrated a GitHub Actions workflow to run the pytest suite automatically on every pull request. This workflow acts as an additional quality gate, providing a status check that must pass before the code can be merged. By running tests in an automated environment, we

catch integration issues and test failures early in the development cycle, long before they could reach production.

Data testing

Implementing data validation with Great Expectations will also be crucial for ensuring the quality and integrity of the data powering our system. Data quality is paramount for any machine learning system; a model is only as good as the data it's trained on.

Six expectations were defined over our processed input JSONL files:

- All JSONs must have a “resume” key containing the description of the resume.
- The “resume” value of all JSONs must have a value between 1 and 8000 characters.
- All JSONs must have a “criteria” key containing an extracted criteria from the resume.
- The “criteria” key must have a value between 1 and 100 characters.
- All JSONs must have a numerical type “score” value.
- The “score” value must be between 0 and 10.

```
-✅ Resume must exist
-❌ Max length of resumes
Analyzed rows: 4410
Percentage of non-compliant data: 10.5%
Percentage of empty data: 0.0%

-✅ Score types
-✅ Score between range
-✅ Criteria must exist
-✅ Max length of criteria
```

Figure 8: Example output after running the Great Expectations pipeline on our resumes processed dataset.

Appendix A: Custom metrics for Criteria Extraction

One of the first steps when training any machine learning system is to define the loss functions that are to be minimized or studied. In the typical ML problems (classification and regression), one commonly uses easily-defined loss functions, such as Mean Square Error (for regression) or Categorical Cross Entropy (for classification). When the task at hand is to generate text, other more convoluted algorithms exist to determine whether the text is close enough to the expected output. However, in our case, it's not a case of text similarities, but rather we need to compare two lists of generated elements. This poses three problems:

1. Elements referring to the same criteria might be expressed with different words in the target and the output.
2. Elements from one list might not be present in the other.
3. Elements from the target list might appear repeated in the outputs.

For this, two bespoke evaluators had to be defined. They both make use of Language Models as Judges to perform the matching between “target criteria” and “predicted criteria”, and here's how they do it:

- **Target Recall:** A large embedding model is used to create vectorial representations of the titles of all the criteria in the target list, as well as the output list. Then, for each element of the output list, the cosine similarity between it and all the target criteria's

embeddings is computed. Third, for each target criteria, the maximum of the cosine similarities is extracted, and a threshold is applied to it: if it is above 0.8, the target is considered as “recalled”, otherwise it’s “not-recalled”. Then, the average recall is computed by casting every similarity under 0.8 to 0, and computing the mean of all the recall similarities.

- **Importance MSE:** A large embedding model is used to create vectorial representations of the titles of all the criteria in the target list, as well as the output list. Then, for each element of the output list, the system discovers the target criterion that it’s most probably referring to by computing the cosine similarity between it and all the target criteria’s embeddings. An output criterion is considered “matched” if it has a cosine similarity to any target criterion above some threshold (in our experiments, 0.5). Then, the algorithm computes the square error between the predicted importance of each criterion and the baseline, target, importance. If a response criterion is not matched, its square error is set to the maximum value it takes. Finally, the average of all the square errors is computed.

The implementation of these two scoring functions can be found in the repository, under [the evaluation script](#).